SWEN30006 Project 2: PacMan in the TorusVerse
Workshop: Wednesday 4:15pm | Team 4
Group Members: Sanskar Agarwal (908069), Elise Marcun (1173017), Lara Stebbens (1187048)

Analysis of Current Design and Initial Refactoring

The current design, as provided, for the simple PacMan game had several areas of concern which would hinder the extension and development of new features when developing another variation of the game: *Pacman in the TorusVerse. Pacman in the TorusVerse* required numerous new features to extend from the basic version, including multiple levels, portals for the pacman and monsters to teleport through, a smarter auto-player strategy for the pacman, as well as level and game checking and editing. These new features would have been difficult to implement and extend upon with the provided code base, therefore, our team decided to refactor the code base to increase its extensibility and maintainability as we implemented new features.

In the original design, the Game class had become overly bloated, complex and difficult to read and comprehend. This would prove difficult to extend when introducing the new features of the TorusVerse version such as multiple levels, and portals for the PacActor and Monsters to teleport through. Therefore, we introduced a MonsterFacade and ItemHandler, similar to in Project 1, to delegate some of this logic to other classes. This would therefore make it easier for the implementation of the smart auto player and portal functionality. Additionally, in the original design the Monster class was being used as a generic object for all types of Monsters, resulting in low extendibility. While this was manageable for the implementation of *Pacman in the TorusVerse*, which only has two monsters, the Tx5 and Troll, this logic would have been difficult to continue when A24 eventually want to extend *Pacman in the TorusVerse* with the functionality and additional monsters from the *Pacman in the Multiverse* version, especially with the added complexity of monster teleportation via portals. Please note that we also grouped functionality by folders within the src/pacman folder to improve readability of the various classes.

The refactoring that we have completed is exhibited in the below Static Design Model (Model 2).

Part 1: Design of Editor
Please refer to the Model 3: State Machine Model

a) <u>Different Modes: Edit and Test Mode</u>
The updated game has different modes: *edit mode* and *test mode*. Edit mode corresponds to the original 2D-Map-Editor package, which allows users to interactively generate, edit and save game maps as XML files, whereas Test mode corresponds to the original pacman package, allowing users to run a game, subject to each level and the game itself passing a series of checks (detailed later). The original code base had these two modes in separate packages, meaning that only one mode could be run at once and that users could not readily switch between the two modes. However, in the updated version, when test mode concludes, whether that be by failing game or level checking, running into a monster or successfully completing the game, the edit mode window should automatically open. This meant that

the two packages needed to be merged into one, with one single Driver. To do so, we put the two main 2D-Map-Editor source files 'editor' and 'grid' inside the pacman 'src' directory in a folder 'mapeditor' to separate the logic from the main pacman game, and merged 2D-Map-Editor's Main class functionality into the pacman's Driver class main method. We also moved all the pacman related classes into a separate folder within 'src' called 'pacman' to maintain this separation.

b) Multiple Levels

In the new version of the game, once the user completes one level by eating all the gold and pills on the map, they should progress to the next level in the specified game folder. Given that (1) setting up a level in the new version follows the same logic as the initial game set up process in the original version; and (2) progressing to the next level (or failing a level) coincides with the original version's end-of-game logic, we determined that it would be best to retain all the level logic in the Game class rather than introducing a new class or interface. This approach reduced code duplication, as we didn't have to replicate the level-specific logic in a separate class, whereas introducing a new class or interface for the levels would increase the complexity of the game and potentially make it more difficult to maintain. It is important to note that our decision to keep all the added level logic in the Game class does have the trade-off of increasing the size of the Game class, which can potentially impact the overall readability of the code. However, we mitigated this by breaking down the original methods in the Game class into smaller units, so that methods could be reused between levels, hence reducing code duplication and increasing maintainability whilst retaining clarity and modularity within the class.

c) Game Checking Functionality

Another new requirement of the updated version was to perform *game checking* to check the validity of the game folder which stores all the game's levels. The two primary checks undertaken were (1) that there is at least one correctly named map file in the folder, and (2) that there is only one map file named with a particular level number. Since there were only two checks required, which both had relatively low complexity and followed similar logic to the logic used when loading the files and levels to set up the game, we decided it was best practice to keep the game checking functionality in the Game class. Similar to our explanation described in 1b), creating a separate class for game checking would increase complexity, code duplication and hence the maintainability of existing code. Additionally, if we created a separate class for game checking, it would have been tightly coupled with the Game class in order to access the necessary game folder data and methods required to perform the checks, and introduced an additional layer of abstraction and complication. However, it is also important to note the trade-offs of our decision. Keeping the game checking logic within the Game class may inhibit extendibility if more game checks were to be implemented, as well as reducing the separation of concerns as the Game class becomes less focused on its primary responsibility of managing the game and level behaviour. However, based on the small number of low complex tests, we decided that it would be better practice to keep all the game checking logic within the Game class to promote low coupling, code maintainability, reduced duplication and lower complexity.

d) Level Checking Functionality

After game checking is completed, each level in the game folder then goes through another validation process: *level checking.* In this version of the game, there were four tests: (1) exactly one starting point for PacMan, (2) exactly two of each portal colour, (3) at least two gold and pills in total, and (4) each gold and pill is accessible to PacMan from the starting point, ignoring monsters but accounting for valid portals. As there are more tests, each being more complex than the game checking tests, we decided that it would be better practice to separate the level checking logic from the Game class using the *Strategy Pattern.* As per the strategy pattern, we introduced a common interface called 'LevelCheckStrategy' that declares the checkLevel() method, which is then implemented by four *Concrete Strategies,* corresponding to each of the four tests listed above. Our *Context* class is called 'LevelChecker' which directly creates a list of relevant 'LevelCheckStrategy' objects and iterates through them to execute the four tests as outlined in the specification. The Strategy Pattern means that there is only one call to perform all the level checks in the Game class, promoting code maintainability and understandability, whilst also being highly flexible and extensible should there be any updates to the current level checks or additional level checks required. Additionally, the Strategy Pattern promotes high cohesion and low coupling by encapsulating each level check as a separate strategy. Each concrete strategy class is focused on a specific responsibility, ensuring high cohesion, whilst also being loosely coupled with the Game class, enhancing modularity and maintainability. Another pattern we considered when designing the level checking was the *Template method* which would define the overall algorithm for level checking in an abstract base class, whilst allowing for specific steps to be overridden in derived classes. At first glance, the Template method may seem more intuitive than the Strategy Pattern given level checking tests are majorly uniform across different levels. However, we determined that the Strategy Pattern would be more suitable because it allows different tests to be added dynamically more easily. In our case, test (4) (defined above) is only tested if (1) to (3) do not fail, therefore, test (4) must be added dynamically conditional on the results of tests (1) to (3). This would be more difficult to assess with the Template method, which tends to follow a more rigid set of steps. Additionally, there was not as much benefit associated with the Template method, as our level checks would not need to be overridden, and the Template method is also less extensible and flexible when adding or modifying level checks. After taking all these considerations in account, we decided the Strategy Pattern would be most suitable due to its extensibility, maintainability, high cohesion and low coupling between individual checking strategies and the Game class.

Part 2: Design of AutoPlayer

a) Smart Autoplayer Strategy

In order to implement a smarter autoplayer for the PacActor class, we opted to utilise the Strategy pattern which supports loose coupling (by allowing the PacActor class to depend on an abstract AutoplayerStrategy interface rather than concrete implementations) and high cohesion (by encapsulating each autoplayer behaviour in a separate strategy class). The Strategy pattern proved to be well suited for implementing autoplayer behaviour due to its ability to dynamically select different strategies based on the context. This provides flexibility and extensibility to the PacActor class. By encapsulating the different

autoplayer strategies, such as FindClosestStrategy and PortalStrategy, we are able to easily add or adapt strategies without having to significantly disrupt the existing code in PacActor class. Additionally, the Strategy pattern promotes Information Expert, as each strategy encapsulates a specific autoplayer behaviour and is focused on its specific responsibility - hence, promoting code modularity, maintainability and readability.

We also evaluated the possibility of using the Template Method. THis pattern would have involved defining a base Autoplayer class that outlines the overall autoplayer behaviour, while allowing subclasses to override specific methods for individual strategies. However, the Strategy pattern was preferred due to the need to be able to dynamically select different behaviours.

It is important to consider that the Strategy pattern introduces some complexity with the need to manage multiple strategy classes and their interactions. It may also incur some runtime overhead due to the need for dynamic strategy selection. The Template Method pattern, on the other hand, provides a more straightforward and structured approach but may be less flexible in terms of behaviour customization and extension.

The strategy pattern is especially useful in the context of enabling the following: eating an ice cube freezes the monsters for a defined duration. To implement further complexity into the autoplayer PacActor, this would require an additional strategy class, we would introduce a new strategy class called IceStrategy that implements the AutoplayerStrategy interface. This IceStrategy will handle the behaviour of the pacActor in autoplayer mode when it eats an ice cube.

b) <u>Teleportation Functionality</u>

The final requirement of the new *Pacman in the TorusVerse* version was to implement portals, allowing for the PacMan and monsters to teleport from one portal pair to another. Adding these new objects, with complex functionality required best practice design to ensure we could sufficiently test their functionality, easily add new coloured portals, and have different actors interact with them. To do so, we decided to implement the Facade Pattern by creating a class 'PortalFacade' which acts as a single point of contact that is responsible for the collaboration between the 'Portal' class and each actor and the Game class.

The Facade pattern offers several benefits in the context of the implemented Portal functionality. Firstly, it promotes high cohesion by providing a single, focused interface (PortalFacade) that encapsulates and coordinates the complex functionality of the Portal class and interactions with other actors. This enhances code readability and maintainability.

The pattern also simplifies the interface between the Game, PacActor and Monster classes. By hiding the complexity of the underlying classes, the PortalFacade acts as a single point of contact, making it easier for these entities to interact with portals. Facade pattern also encapsulates the interactions and dependencies between the Portal class, actors, and the Game class within the PortalFacade. This

improves code organisation and reduces coupling between different components and promotes modular design principles.
The Facade pattern also promotes loose coupling between components by abstracting the complexity of the underlying subsystems (Portals) and providing a simplified interface (PortalFacade) to the clients (actors and Game class). This reduces dependencies and allows for easier modifications and updates.

The introduction of a level of indirection is another advantage of the Facade pattern. The PortalFacade acts as an intermediary, shielding clients from the complexities of the Portal class and providing a simplified interface. This simplifies the overall system structure, enhances flexibility and allows for easier maintenance.

The Facade pattern helps protect variations and changes within the Portal subsystem. The PortalFacade acts as a buffer, isolating the clients from the internal details of the Portal class. This protects against variations and changes within the Portal subsystem, allowing modifications to be made to the Portal class without affecting the clients or the Game class.

Moreover, the use of the Facade pattern promotes flexibility and extensibility. The implementation of the PortalFacade class allows for easy extension and modification of the portal functionality. The use of an enum for portal colours and a method to return the file path of their corresponding images enhances flexibility. This modularity and maintainability of the codebase enables the addition of new coloured portals without affecting other actors or the game class.

However, it is also important to consider that the introduction of the Facade pattern adds an additional layer of abstraction and complexity to the system. Additionally, the Facade pattern provides a single, simplified interface which may limit the level of customisation. Also note, that we did not implement Singleton for the Facades because the multiple levels requires a new PortalFacade to be created for each grid.

Assumptions
Please note that the following assumptions were made about the specification

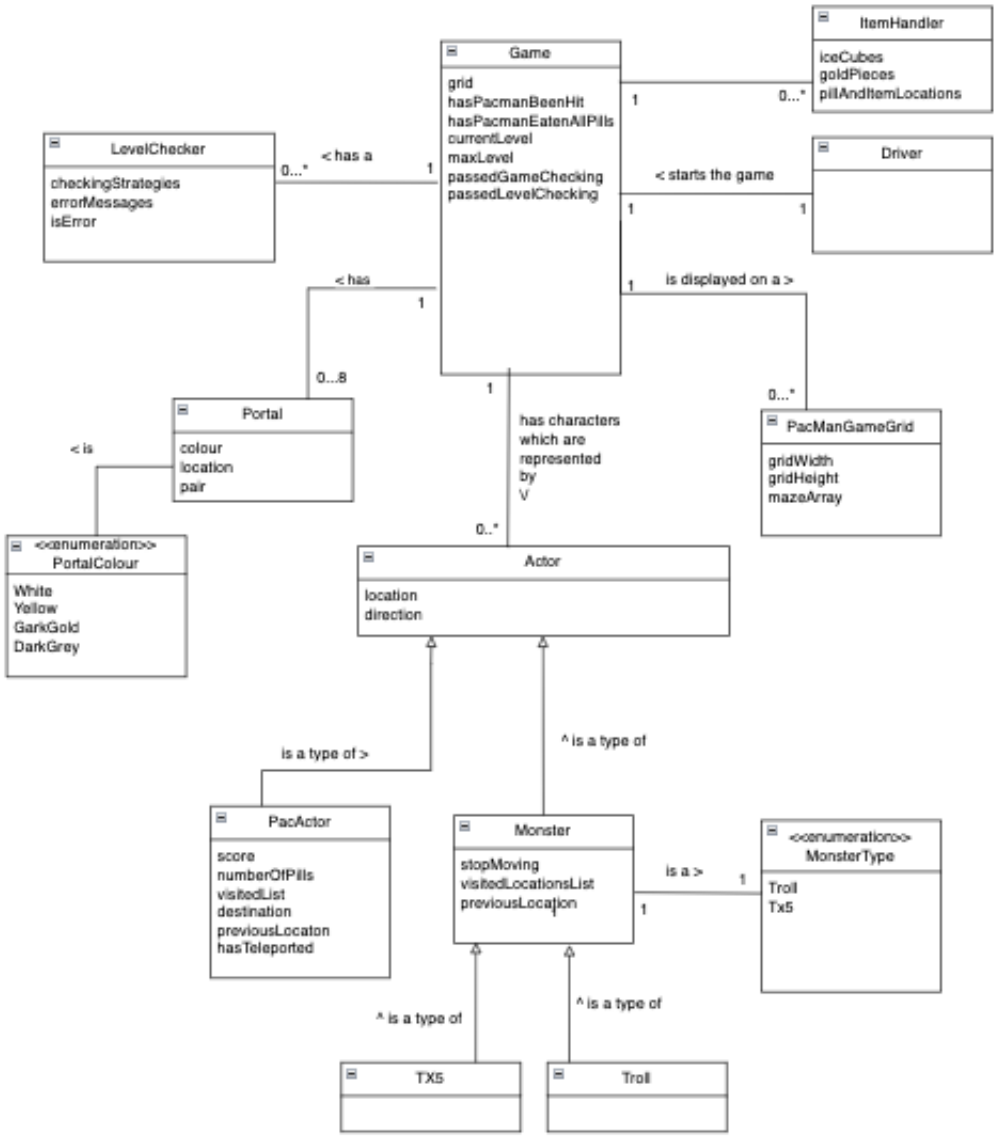
- On Ed, it was established that we were not required to implement the functionality of the start button
- As discussed with Charlie Ding (our tutor), occasionally our Pacman and a Monster will appear to have collided, however they have effectively switched position and were never on the same cell location as each other at the same time. This is not deemed to be a collision.
- In the case that both game and level checking fails, the log will print out all errors and the editor will open a blank editor, not specific one

Appendix: Software Models

Please use this link to refer to the Software Models in drawio for improved readability:
https://drive.google.com/file/d/1w-JO36SjKu4MLnCQN4lva1PSmbY-TcDr/view?usp=sharing

You will need to log in with your University of Melbourne email address to access the link due to UniMelb drive sharing restrictions.

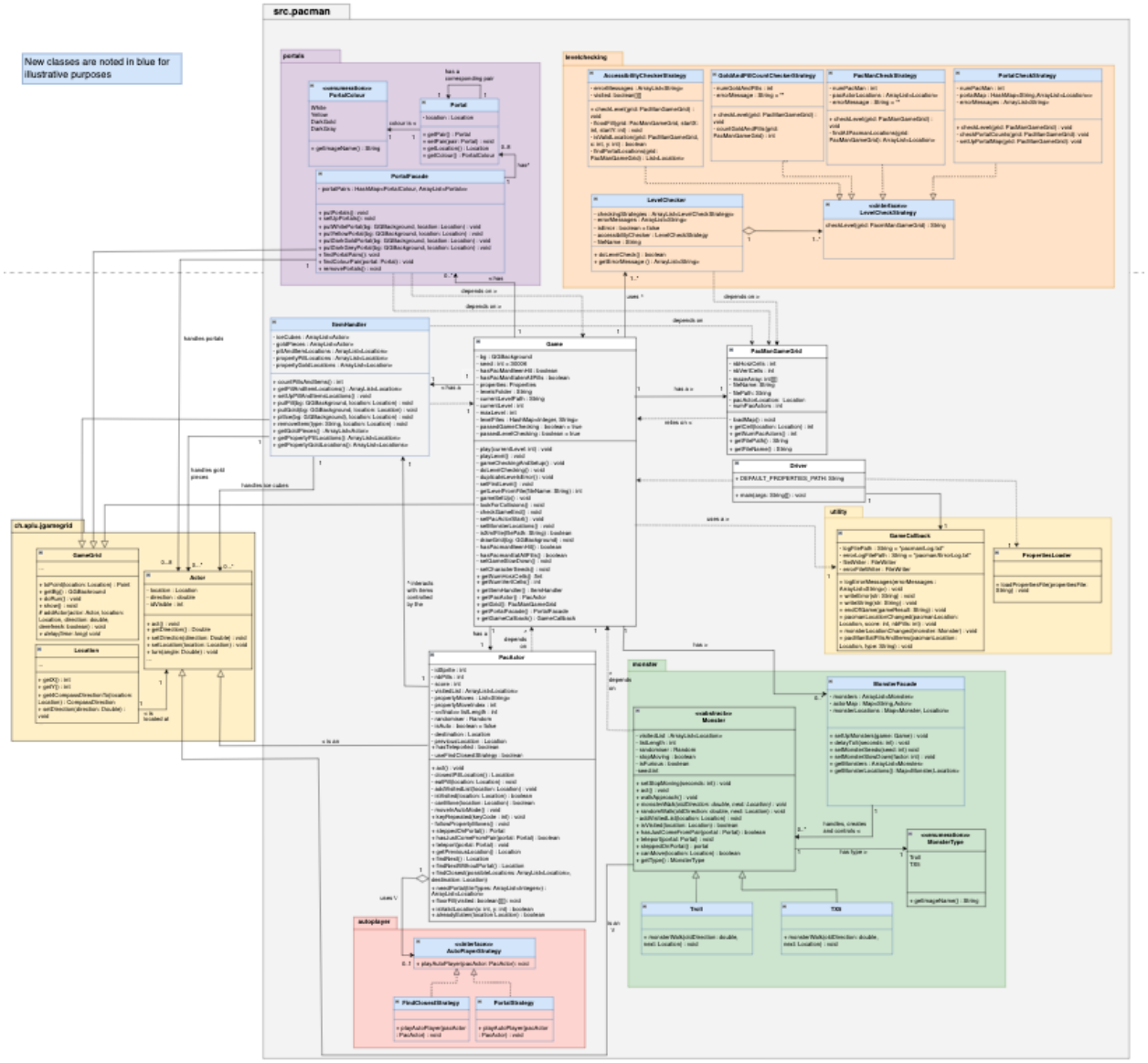
Model 1: Domain Class Diagram

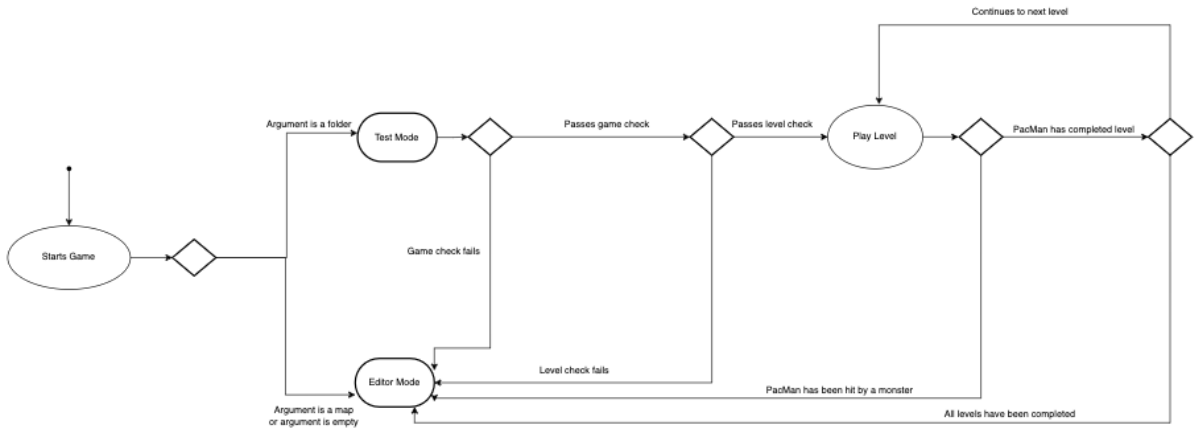


Note: Due to the new game modifications where there are multiple levels, there can now be multiple PacManGameGrids per game, as well as multiple ItemHandlers and LevelCheckers. This is exhibited in the multiplicities in the Design and Domain model.

Model 2: Static Design Model

Please note that this Design Model is not exhaustive of all getters and setters. Most getters and setters are present, however to preserve space and improve readability we have had to exclude some less 'important' ones. All other relevant methods are included.



Blue classes represent new classes we have added in the updated version of the game. The yellow folders are existing folders from the base code, whereas we have added the new coloured folders to help improve understanding.

Model 3: State Machine Model